

CoTC-SpMM: A Cooperative Tensor-CUDA Cores Scheme for Efficient Sparse Matrix Multiplication

Qin Du
The College of Computer Science and
Electronic Engineering
Hunan University
Changsha, China
seery313@hnu.edu.cn

Shengle Lin*
The College of Computer Science and
Electronic Engineering
Hunan University
Changsha, China
lsl036@hnu.edu.cn

Yuedan Chen
The Big Data Institute
Central South University
Changsha, China
chenyuedan@csu.edu.cn

Guoqing Xiao*
The College of Computer Science and
Electronic Engineering
Hunan University
Changsha, China
xiaoguoqing@hnu.edu.cn

Marc Casas
Barcelona Supercomputing Center
Barcelona, Spain
Polytechnic University of Catalonia
Barcelona, Spain
marc.casas@bsc.es

Kenli Li
The College of Computer Science and
Electronic Engineering
Hunan University
Changsha, China
lkl@hnu.edu.cn

Abstract

Sparse Matrix-Matrix Multiplication (SpMM) is a fundamental operation in scientific computing and artificial intelligence. However, the inherent sparsity and irregularity of real-world datasets present significant challenges for developing high-performance SpMM kernels on modern GPUs. Existing approaches typically focus on a single type of compute unit, leaving the potential for cooperative parallelism between heterogeneous GPU cores largely under-explored. This paper introduces **CoTC-SpMM**, a cooperative Tensor-CUDA cores scheme for efficient SpMM on GPUs. Specifically, we first propose the HTC format to partition sparse matrices into dense and sparse components, enabling specialized kernels to leverage the distinct advantages of different computing units and maximize hardware utilization. Moreover, we implement several low-level runtime optimizations, including 1-D resource mapping for load balancing, software pipelining to hide memory latency, and PTX-level instruction tuning to enhance SpMM throughput. Experimental results on NVIDIA A100 and H800 GPUs demonstrate that CoTC-SpMM achieves substantial performance speedups over state-of-the-art implementations.

CCS Concepts

• Computing methodologies → Parallel algorithms.

Keywords

SpMM, GPU acceleration, cooperate TCUs and CUDA cores

ACM Reference Format:

Qin Du, Shengle Lin, Yuedan Chen, Guoqing Xiao*, Marc Casas, and Kenli Li. 2026. CoTC-SpMM: A Cooperative Tensor-CUDA Cores Scheme for

*Corresponding author



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

ICPP '26, Singapore, Singapore

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2657-6/2026/09

<https://doi.org/10.1145/3832810.3832857>

Efficient Sparse Matrix Multiplication. In *Proceedings of the 55th International Conference on Parallel Processing (ICPP '26), September 28-October 01, 2026, Singapore, Singapore*. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3832810.3832857>

1 Introduction

Sparse Matrix-Matrix Multiplication (SpMM) optimization has been a persistent research focus [12, 27]. SpMM is a critical sparse operator that computes a dense matrix C by multiplying a sparse matrix A with a dense matrix B . As a crucial sparse operation, SpMM has been extensively applied in the fields of scientific computing and artificial intelligence [1, 2, 9, 23].

A large body of research has explored GPU-based acceleration strategies for SpMM. For example, *ASPT*[12] designs an adaptive tiling strategy to improve dense data locality through column re-ordering. *Sputnik*[8] proposes a 1-D tiling scheme and the ROMA technique to achieve load balance. Furthermore, *RoDe*[18] improves memory throughput by partitioning rows into regular blocks and residual segments. While these existing approaches primarily target standard CUDA cores, the advent of specialized **Tensor Cores (TCs)** provides a new frontier for further accelerating SpMM.

However, TCs are originally designed for fixed-size dense block operations. When applied directly to sparse computations, naive TC-based strategies often yield suboptimal performance due to low hardware utilization. To better exploit the computational power of TCs, researchers have investigated structural compression techniques to minimize redundant operations. For instance, *TC-SpMM* [26] introduces sparse graph translation, which extracts non-zero vectors from sparse matrices and merges them into TC blocks. Building upon this, *DTC-SpMM* [7] implements a low-level PTX kernel with further optimizations. *FlashSparse*[19] reduces memory footprint by minimizing sparsity granularity via a swap-and-transpose strategy, while *BRP-SpMM*[5] adopts a block-row partitioning method to separately utilize Tensor and CUDA cores.

Despite these advances, several challenges remain unresolved. First, most existing works optimize only a single type of compute unit, neglecting the potential for cooperative parallelism between Tensor and CUDA cores. Second, state-of-the-art implementations

Algorithm 1 Naive SpMM Calculation Process**Require:** Sparse matrix $A_{m \times k}$ Dense matrix $B_{k \times n}$;**Ensure:** Dense matrix $C_{m \times n}$.

```

1: for  $z = 1 \rightarrow N$  do
2:   for  $i = 1 \rightarrow M$  do
3:     for  $j = 1 \rightarrow K$  do
4:       if  $A[i][j] \in \text{NNZ}$  then
5:          $C[i][z] += A[i][j] \times B[j][z]$ 
6:       end if
7:     end for
8:   end for
9: end for

```

often rely on costly matrix reordering to achieve performance gains, which limits their scalability and practical utility. Third, there remains a critical need for a lightweight yet efficient GPU runtime kernel capable of adapting to diverse sparsity patterns across scientific computing and AI workloads.

To this end, we propose a Cooperative Tensor-CUDA cores scheme for fast SpMM on GPUs, called **CoTC-SpMM**¹. We first introduce a Hybrid Tensor-CUDA (HTC) sparse format which can partition a sparse matrix into TC blocks and a residual CUDA part. Furthermore, we design two highly optimized runtime kernels that maximize inter-core parallelism through fine-grained workload coordination between TCs and CUDA cores. Extensive experiments on SuiteSparse and pruned ResNet50 matrices reveal that CoTC-SpMM achieves average speedups of 3.07×, 2.62×, 1.78×, and 1.21× over cuSparse, DTC-SpMM, RoDe, and FlashSparse, respectively.

In summary, the main contributions of this paper are as follows:

- We propose CoTC-SpMM, a novel cooperative parallel computing scheme that jointly exploits TCs and CUDA cores to accelerate SpMM on GPUs.
- We design HTC format, a hybrid sparse format featuring efficient compression and partitioning mechanisms, specifically tailored to the computational characteristics of different GPU compute units.
- Extensive experiments on A100 and H800 GPUs demonstrate that CoTC-SpMM achieves significant speedups over multiple state-of-the-art works across diverse datasets.

2 Background and Motivation

2.1 SpMM Based on Tensor Core

Sparse Matrix-Matrix Multiplication (SpMM) is a crucial sparse operator that can be formulated as $C_{M \times N} = A_{M \times K} \times B_{K \times N}$. Its naive computing process is shown in Algorithm 1.

While the basic algorithm of SpMM is straightforward, achieving high performance on modern GPUs requires careful exploitation of matrix multiplication primitives and hardware acceleration mechanisms. For instance, NVIDIA provides two primary APIs for matrix multiplication at the PTX level: WMMA and MMA. WMMA supports limited matrix shapes and imposes strict constraints on data layout, whereas MMA offers greater flexibility by allowing developers to explicitly control data distribution among threads

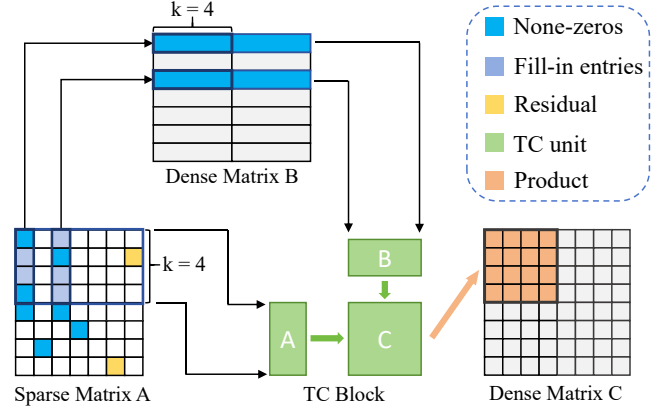


Figure 1: SpMM computation on a TC block, assuming the matrix fragments are 4×2 for A, and 2×4 for B.

and manage data loading and storage operations. The Compressed Sparse Row (CSR) [21] format is the most common storage scheme for sparse matrices. However, its unstructured data layout creates a significant architectural mismatch with the rigid, regular computation fragments required by Matrix Multiply-Accumulate (MMA) instructions. As a result, attempting to align arbitrary non-zero elements into MMA-compatible operand shapes often results in excessive resource under-utilization and computational waste. To improve the computational utilization of MMA instructions, existing studies [14, 17] have proposed an efficient sparse data storage scheme called the TC block. As shown in Fig.1, when executing SpMM via MMA instructions, a TC block is constructed by packing non-zero columns from matrix A and fetching the corresponding dense rows from matrix B to perform the computation.

2.2 Computational Redundancy of TC Blocks

To ensure the correctness of MMA instruction execution, additional zero padding must be applied within the TC block. However, due to the high sparsity of real-world sparse matrices, only a small proportion of the elements in the TC block are effective calculation entries. Previous work adopted various vector sizes to identify non-zero columns. For example, DTC-SpMM [7] uses vectors of length 16, while FlashSparse [19] adopts length-8 vectors. For sparse matrices with different sparsity patterns, Fig. 2 demonstrate the Relative Computational Workload (RCW) for different TC-based SpMM frameworks. The RCW is defined as the ratio of total workload to effective workload. As shown in the figure, the computational redundancy of TC-based SpMM gradually decreases as the vector length becomes finer, but a substantial portion of computing power continues to be wasted. In contrast, CoTC-SpMM coordinates the workload between TC and CUDA core to perform zero padding as needed, thereby achieving a significant advantage in overall workload compared to solely TC-based approaches. We observe that the DTC-SpMM and FlashSparse framework increase redundant computation by factors of 4.41× and 3.28× compare to the baseline, respectively, while our method achieves an average redundancy ratio of only 1.41×.

¹<https://github.com/seery-cmd/CoTC-SpMM>

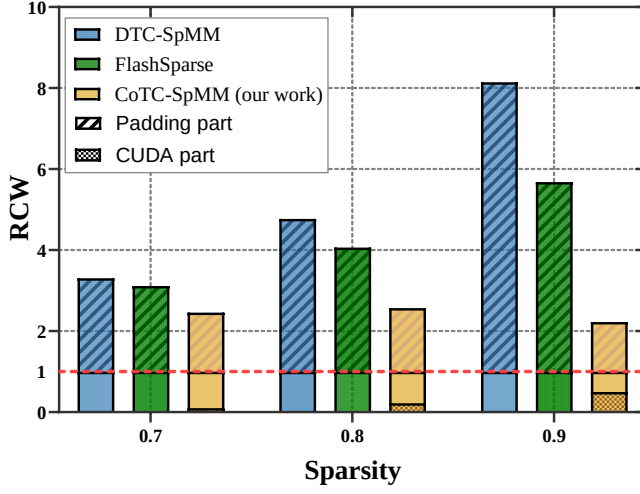


Figure 2: The relative computational workload under different TC-based SpMM framework. Note that, the region under the red line represents the the effective calculation part of TC block, while the upper region represents the invalid calculation part of padding zero.

Table 1: The average clock cycle under different mma shapes.

shape	load A	load B	MMA	Total
m16n8k8	22	1884	20	1926
m8n8k4	37	608	1955	2600

2.3 Latency of Discrete Memory Accesses

In practice, the sparse matrix A and the dense matrix B are typically stored in row-major order. When performing TC based MMA calculations on the TC block storage format, discrete access to dense data becomes unavoidable. Meanwhile, the MMA instruction provides different fragment shapes that arrange the tiles of matrix B in distinct memory layouts. For instance, Chen[3] employed a row-major MMA with **m8n8k4** (where a warp executes 4 MMA operations), whereas the state-of-the-art work *Flashsparse*[19] adopted a column-major MMA with **m16n8k8** (where a warp executes 1 MMA operation). Their corresponding data layouts are illustrated in Fig. 3. Although these MMA shapes perform mathematically identical computations, significant performance discrepancies arise at different computation stages in SpMM workloads. To illustrate this phenomenon, we compare the clock cycles of each step on the matrix *wv2010* which exhibits a uniform row distribution. As shown in Table. 1, the total cost of m16n8k8 is lower than m8n8k4, yet the strided-access to matrix B under m16n8k8 is approximately 3× slower than the sequential access in m8n8k4. The memory access latency of SpMM is sensitive to the data layout of TC blocks, thus must be carefully considered during optimization.

2.4 Abundant Computing Units on GPUs

Moreover, each Streaming Multiprocessor (SM) in NVIDIA GPUs integrates multiple functional units, including Tensor cores, CUDA cores, registers, warp scheduler, etc. Most existing works focus on

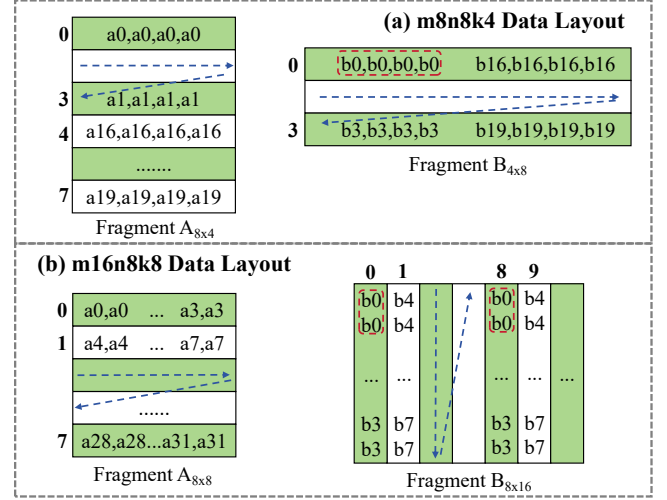


Figure 3: (a) represents the data layout of the MMA with m8n8k4 and (b) represents the data layout of the MMA with m16n8k8 after transposition.

optimizing a single type of compute unit, which limits their ability to fully exploit the parallel computing potential of SMs when handling sparse workloads across diverse scenarios [11, 13, 20]. When the matrix size is large, the utilization of SM resources tends to approach saturation; in this case, leveraging Tensor cores, which offer higher computational throughput, yields the best efficiency. Conversely, for smaller matrices, the SM can accommodate a larger number of parallel tasks, and cooperative execution across different computing units can significantly improve overall resource utilization. As shown in Fig. 2, non-zero elements that cannot be organized into a TC block can be processed by CUDA cores. This parallel strategy reduces TC workload while boosting overall throughput via reasonable task partitioning.

2.5 Motivation

The above analysis reveals four major performance challenges that persist in current SpMM research: ① The irregular memory access patterns to matrices A and B that we discuss in Section 2.1 remain a bottleneck that limits the performance of SpMM on GPUs. ② Following the analysis in Section 2.2, there remains room for optimization in reducing the computational redundancy caused by zero padding without incurring performance loss. ③ As described in Section 2.3, there continues to exist considerable potential for further optimizing the data loading efficiency of matrix B under the m16n8k8 configuration. ④ Given the diverse sparsity characteristics of real-world matrices, it is crucial to develop a flexible optimization strategy capable of fully harnessing the computational power of SMs through heterogeneous cooperation. Motivated by these observations, we propose the CoTC-SpMM, a high-performance SpMM scheme that integrates Tensor cores and CUDA cores for cooperative computation, while incorporating an efficient pipelined execution mechanism to effectively overlap MMA computations with memory access latency.

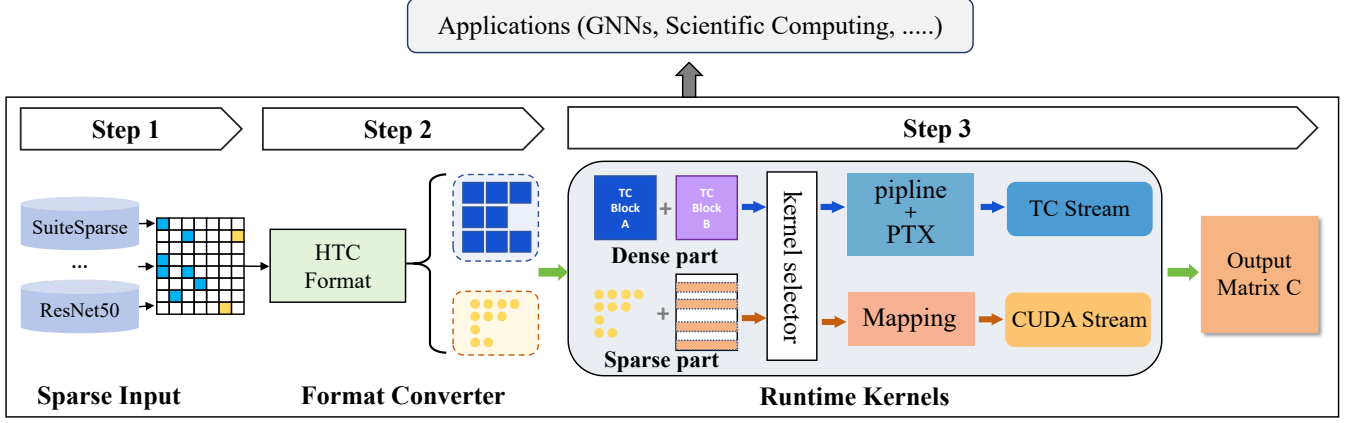


Figure 4: An overview of CoTC-SpMM framework.

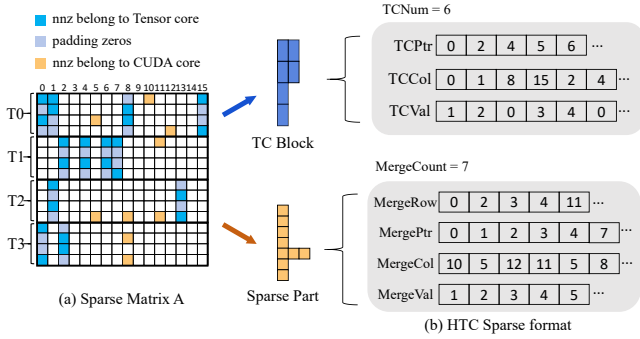


Figure 5: An example of Hybrid Tensor-CUDA (HTC) format.

3 CoTC-SpMM Design

3.1 Overview

As illustrated in Fig. 4, CoTC-SpMM consists of several key components: (1) The Hybrid Tensor-CUDA (HTC) sparse format converter, which divides the sparse input into dense and sparse parts based on a predefined threshold α . (2) The Runtime Kernels that incorporate a series of optimization techniques, including: ① A highly optimized PTX-level Tensor Core (TC) computation stream employing a dual-buffer pipelined execution scheme based on shared memory; ② A CUDA Core computation stream guided by an efficient data-mapping strategy; ③ A kernel selector that chooses the appropriate kernel according to the sparsity of input. The HTC data layout can offer high scalability and remain compatible with both TC-based and CUDA-based optimization schemes. Within the runtime kernel, two distinct parallel strategies are integrated to coordinate TC and CUDA cores, enabling efficient SpMM across different scenarios.

3.2 Hybrid Tensor-CUDA Sparse Format

To enable irregular memory access patterns for both TCs and CUDA cores, we propose the HTC sparse format that classifies the columns of matrix A into dense and sparse categories. Fig. 5 provides an example of CoTC-SpMM using CUDA parallelism to compress sparse matrices into the proposed HTC format. Specifically, an $n \times 1$ column

with fewer than α nonzero elements is treated as sparse, whereas those with at least α nonzeros are classified as dense. HTC collects and assembles dense columns into TC-friendly blocks, padded with zeros when necessary. The remaining columns that cannot be assembled into complete TC blocks are reclassified as sparse. In practice, the TC block size is set to 8×8 in our experiments. Each thread processes row fragments of size 8 and aggregates the TC blocks into a block group. Finally, HTC format stores the total number of TC blocks in *TCNum*, and records row offsets, column indices, and element values in three arrays: *TCPtr*, *TCCol*, and *TCVal*.

The remaining sparse elements are stored in a compact form inspired by CSR format. In practical matrices, sparse regions often contain many empty rows, incurring avoidable memory overhead. To address this, HTC further compresses the residual part by removing empty rows. Specifically, HTC removes blank segments and stores only effective rows using *MergeCount* and four arrays—*MergeRow*, *MergePtr*, *MergeCol*, and *MergeVal*—to represent the row indices, row pointers, column indices, and values, respectively. The entire format conversion requires only two traversals of the matrix: the first estimates memory requirements, and the second separates dense and sparse partitions, as shown in Algorithm 2. Before performing the thread-level format conversion based on row slices, the column indices within each thread's assigned region are reordered (Alg.2 line 6). Following this, duplicate column indices are removed, and the frequency of each distinct column index is recorded (Alg.2 lines 7-14). Subsequently, *all_sum* is used to tally the dense columns that successfully constitute into a TC block (Alg.2 lines 15-22). The memory requirements of the TC and CUDA parts are determined through atomic accumulation, and are then written to the appropriate memory addresses by thread 0 (Alg.2 lines 23-28). After global memory allocation, the row slice array is traversed again to finalize the partitioning of the data into sparse and dense parts (Alg.2 lines 30-39).

Moreover, the threshold α plays a critical role when sparse workload can not saturate Tensor core. When $\alpha = 1$, the CUDA stream handles only dense columns that fail to form complete TC blocks, resulting in limited parallelism and potentially degraded performance. As α increases, the focus of computation to gradually shifts

Algorithm 2 HTC Sparse Format Conversion**Require:** Sparse matrix A , The threshold α ;**Ensure:** Tensor core part and CUDA core part.

```

1: /* Step1 */
2: __shared__ TC_block_num[1]
3: __shared__ CUDA_block_num[1]
4: start = MtxPtr[threadID * shapeM]
5: end = MtxPtr[(threadID + 1) * shapeM]
6: sort(A.col)
7: for  $j = start \rightarrow end$  do
8:   if  $A.col[j] \neq temp$  then
9:      $temp = A.col[j]$ 
10:     $ptr++ = 1$ 
11:    RecodeVecCol[start + ptr] = A.col[j]
12:   end if
13:   RecodeColNum[start + ptr]++
14: end for
15: for  $j = 0 \rightarrow ptr$  do
16:   if RecodeColNum[start + j] >= ColSize then
17:      $temp\_sum++ = RecodeColNum[start + j]$ 
18:     if get_ColNumber%ShapeN == 0 then
19:        $all\_sum++ = temp\_sum$ 
20:     end if
21:   end if
22: end for
23: atomicAdd(&TC_block_num[0], all_sum)
24: atomicAdd(&CUDA_block_num[0], end - start - all_sum)
25: if threadID == 0 then
26:   nnz_tcore[blockId.x] = TC_block_num[0]
27:   nnz_core[blockId.x] = CUDA_block_num[0]
28: end if
29: /* Step2 */
30: for  $j = 0 \rightarrow ptr$  do
31:   if RecodeColNum[start + Ccount] < ColSize then
32:     cudaColRecord = RecodeVecCol[start + j]
33:   else if RecodeColNum[start + Tcount] >= ColSize then
34:     TCColRecord = RecodeVecCol[start + j]
35:   end if
36: end for
37: for  $j = start \rightarrow end$  do
38:   writeVal(CUDAPart, TCpart)
39: end for
40: Merge(CUDAPart)

```

from Tensor cores to CUDA cores. Selecting an appropriate threshold α to balance the workload across TCs and CUDA cores is a prerequisite for achieving optimal performance. On the NVIDIA A100 GPU, the FP16 floating-point performance of Tensor cores is approximately 4 $\times$ that of CUDA cores. Thus, the optimal theoretical performance is achieved when the ratio of non-zero elements assigned to Tensor cores versus CUDA cores is 4:1. This indicates that each dense column within the TC-processed block must contain at least 2 non-zero elements in an 8×8 TC block. As shown in Fig. 6, the performance of CoTC-SpMM exhibits clear sensitivity to different α settings. Based on extensive empirical evaluation across

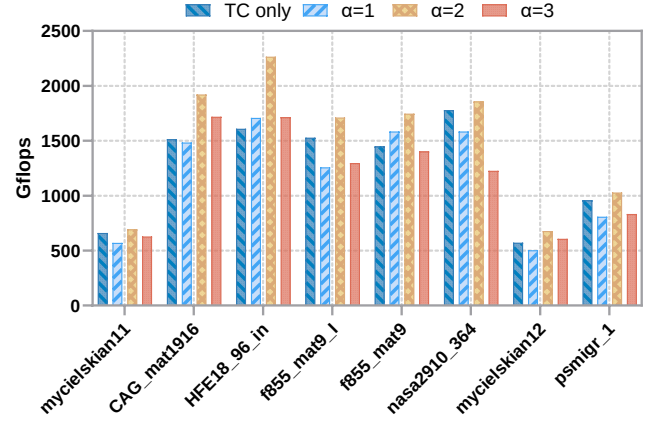


Figure 6: Performance tuning results of CoTC-SpMM under different α factor and TC only on NVIDIA A100 GPU.

various datasets, we set $\alpha = 2$ as the default threshold, achieving an effective balance between Tensor-CUDA workload distribution and overall runtime efficiency. Meanwhile, the selection of threshold α is closely related to the GPU hardware architecture. To ensure scalability across different GPU platforms, we set α as an adjustable hyperparameter.

3.3 The Runtime GPU Kernel Optimizations

CoTC-SpMM integrates two highly optimized GPU kernels — a TC kernel and a CUDA kernel — to efficiently execute SpMM on modern GPUs. The TC kernel adopts a pipelined execution model that overlaps computation with memory operations to hide data-movement latency. In contrast, the CUDA kernel leverages a fine-grained mapping strategy to balance workloads across threads within each block. The runtime kernel pseudo-code is provided in Algorithm 3. To accommodate diverse sparsity patterns, CoTC-SpMM further incorporates a dynamic kernel selector that adaptively chooses the optimal execution strategy, thereby maximizing overall GPU utilization and runtime efficiency.

3.3.1 TC kernel. The TC kernel in CoTC-SpMM employs a single thread block to process all TC blocks within an 8-row slice of matrix A . Each *m16n8k8* PTX instruction operates on an 8×8 TC block from matrix A and an 8×16 dense tile from matrix B to produce an 8×16 temporary dense fragment C . As shown in Fig. 7, assuming the column dimension of matrix B (BN) is 256, each thread transfers 16 bytes from global memory to shared memory using the PTX-level `cp.async` instruction. Since matrix B is stored in row-major layout, each warp loads one contiguous row segment, allowing eight warps to collectively prepare the full input required by a TC block. These segments are placed sequentially in shared memory, after which the PTX-level `ldmatrix.sync` instruction loads them into MMA B fragments. Meanwhile, the TC blocks of sparse matrix A —also stored in row-major order within the HTC format—can be directly loaded into the MMA A fragments without additional transformation. As illustrated in the pipeline diagram on the right-hand side of Fig. 7, the `cp.async` stage is the dominant latency contributor during the SpMM computation.

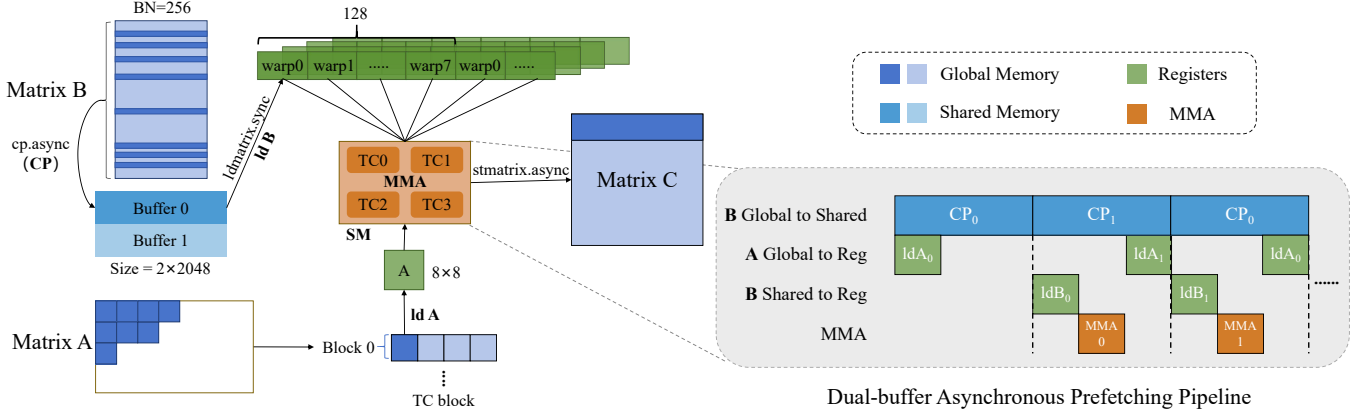


Figure 7: Workflow of the TC kernel (left), and the dual-buffer asynchronous prefetching pipeline strategy (right).

Algorithm 3 CoTC-SpMM Runtime Kernel Design

Require: Sparse matrix A, Dense matrix B;

Ensure: Dense matrix C.

```

1: TC block, Sparse Part = TransHTC(Sparse_A)
2: /* TC Stream */
3: allocate shared_B[2][2048]
4: allocate register_A[1]
5: allocate register_B[4]
6: allocate register_C[4]
7: allocate register_switch
8: shared_B[register_switch] = CP_async(global_B)
9: register_A[0] = global_A
10: for i = rowT_ptr[i] → rowT_ptr[i + 1] do
11:   register_switch ^= 1
12:   shared_B[register_switch] = CP_async(global_B)
13:   CP_ASYNC_WAIT_GROUP(1)
14:   __syncthreads()
15:   register_B[0] = LD_mtxB(shared_B[register_switch])
16:   register_B[2] = LD_mtxB(shared_B[register_switch])
17:   register_C[0] = mma(register_B[0], register_A[0])
18:   register_C[2] = mma(register_B[2], register_A[2])
19:   register_A[0] = global_A
20: end for
21: CP_ASYNC_WAIT_GROUP(0)
22: __syncthreads()
23: register_B[0] = LD_mtxB(shared_B[register_switch])
24: register_B[2] = LD_mtxB(shared_B[register_switch])
25: register_C[0] = mma(register_B[0], register_A[0])
26: register_C[2] = mma(register_B[2], register_A[2])
27: global_C = atomicAdd(register_C)
28: /* CUDA Stream */
29: allocate T_row
30: allocate T_col
31: for i = rowC_ptr[T_row] → rowC_ptr[T_row + 1] do
32:   temp += valA[i] * valB[colIdx[i] * Bn + T_col]
33: end for
34: global_C = atomicAdd(temp)

```

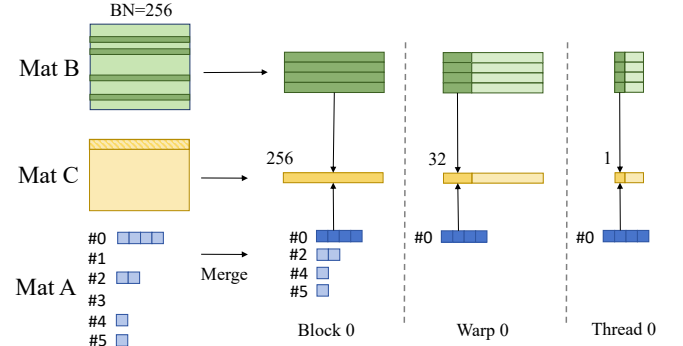


Figure 8: Workflow of CUDA kernel in CoTC-SpMM.

To alleviate the high latency of data transfer between global and shared memory, we design a dual-buffer asynchronous prefetching pipeline leveraging shared memory. This strategy overlaps the MMA computation and fragment loading with the prefetching of subsequent *B* rows, thereby improving Tensor core utilization and overall throughput. During each MMA iteration, all warps operate on the same *A* fragment while consuming their corresponding *B* fragments. Each warp invokes the *m16n8k8* PTX instruction twice to compute two *C* fragments, which are accumulated in registers across iterations until the entire 8-row slice is completed. Finally, the accumulated *C* fragments are written back to global memory via the PTX-level *stmatrix.sync* instruction.

3.3.2 CUDA kernel. The CUDA kernel in CoTC-SpMM employs a 1-D mapping strategy, where each thread block is mapped to an entire row of the dense matrix *B*. As illustrated in Fig. 8, the HTC partitioning process leaves the CUDA cores to process relatively few non-zero elements per row. This sparsity reduces the workload per thread in the thread block, allowing for a higher degree of parallel execution. Therefore, each thread block focuses on parallelizing computations on *B*, preserving contiguous memory access and improving load efficiency. Workload balancing between CUDA cores and TCs must also account for hardware constraints, notably the limited register resources available per SM. Additionally, the

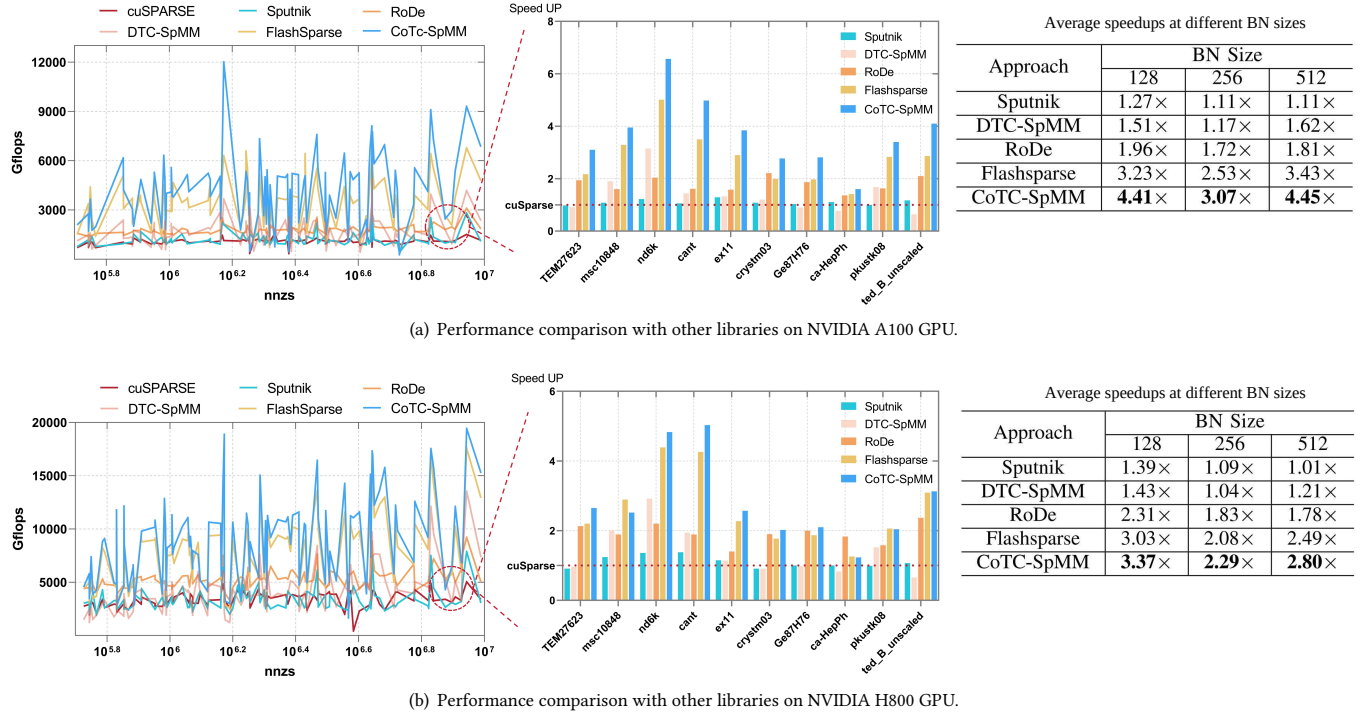


Figure 9: Performance results of various SpMM frameworks under FP16 precision. The left panel reports the throughput on 279 SuiteSparse matrices with BN (the column dimension of matrix B) fixed at 256. The central panel presents the speedup over cuSparse on ten randomly selected representative matrices. The right table shows the average speedups over cuSparse.

merge operation can further reduce the number of thread blocks dispatched to CUDA cores, eliminating redundant inspection overhead. Within a block, each thread performs fused multiply-add (FMA) operations on its assigned nonzero elements and the corresponding dense values based on thread ID. Partial results are accumulated in registers and subsequently written back to global memory. Because all required data within a block can be processed in a single pass, additional pipelining provides little benefit. Instead, we apply loop unrolling to increase instruction-level parallelism and thereby enhance overall throughput.

3.3.3 Kernel Selector. CoTC-SpMM incorporates a lightweight kernel selector that dynamically chooses between two execution strategies: **TC only** and **Combined**. Specifically, it determines whether the sparse workload is sufficient to saturate Tensor core execution (TC only) or whether additional CUDA-core parallelism is required to maximize SM utilization (Combined). The selector determines the occupancy of SM for the runtime kernel based on various factors, including the global characteristics of the sparse matrix and the properties of the underlying hardware architecture.

Based on the HTC format representation of sparse matrices, the total number of thread blocks required by CoTC-SpMM can be expressed as:

$$TB_{htc} = \text{mtx.row/window} + \text{merge_count}, \quad (1)$$

where the first term corresponds to the thread blocks generated from dense part, as each thread block processes matrix slices of

a specified *window* size. The second term reflects the number of CUDA-side merge segments.

Moreover, three factors affect the SM occupancy: the total number of registers per SM, the shared memory size, and the thread block size. Based on these factors, we define β as the theoretical maximum SM occupancy of CoTC-SpMM runtime kernel, which is calculated as follows:

$$\beta = \min(R_{\max}/R_t, S_{\max}/S_t, T_{\max}/T_t), \quad (2)$$

where R_t , S_t , and T_t denote the total number of registers required by the thread block, the shared memory size, and the number of parallel threads in the thread block, respectively. Meanwhile, the maximum number of concurrently executable thread blocks on the GPU is estimated as:

$$TB_{\text{satrate}} = \beta * SM_{\text{count}}. \quad (3)$$

Based on these settings, our kernel selector chooses the appropriate execution mode through the decision rule:

$$\text{CoTC-SpMM} = \begin{cases} TB_{htc} > TB_{\text{satrate}}, & \text{TC only;} \\ TB_{htc} < TB_{\text{satrate}}, & \text{Combined.} \end{cases} \quad (4)$$

In essence, the kernel selector balances the workloads assigned to Tensor cores and CUDA cores, ensuring that CoTC-SpMM maintains high GPU occupancy and achieves optimal end-to-end performance across diverse sparse matrices.

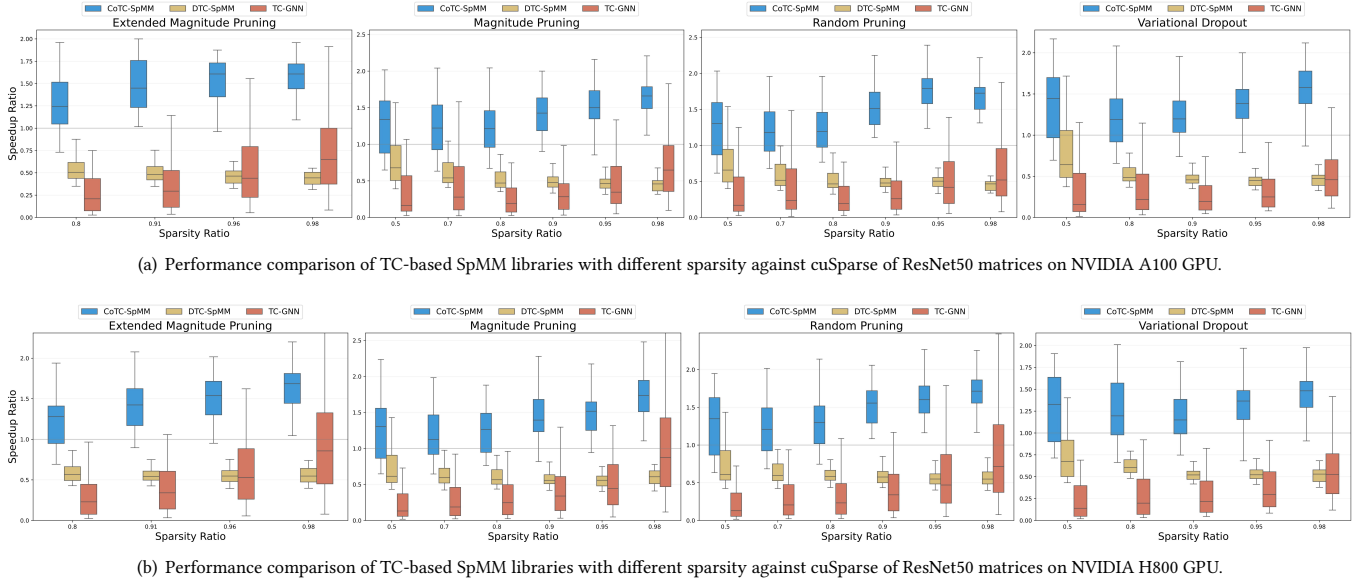


Figure 10: Performance of TC-based SpMM on AI workloads across varying sparsity levels and pruning strategies.

4 Experimental Results

4.1 Experimental Setup

4.1.1 Platforms. We test CoTC-SpMM on two NVIDIA GPUs: A100-PCIE-40GB and H800-SXM-80GB. The A100 GPU is equipped with 108 SMs, 6192 CUDA cores, and 432 Tensor cores, and all kernels are compiled with "nvcc -O2 -gencode arch=compute_80, code=sm_80". The H800 GPU provides 132 SMs, 16896 CUDA cores, and 528 Tensor cores, compiled using "nvcc -O2 -gencode arch=compute_90, code=sm_90".

4.1.2 Benchmarks. We evaluate CoTC-SpMM on three categories of workloads. (I) **Scientific computing:** following the criteria used in FlashSparse, we select 279 square matrices with more than 10K rows from the SuiteSparse Matrix Collection [4]. (II) **Artificial intelligence:** we collect 1130 sparse matrices derived from ResNet50 [10] under magnitude pruning, extended magnitude pruning, random pruning, and variational dropout [30]. (III) **GNN model:** We select 4 graph-structured datasets from TUDataset², following the same selection criteria as HC-SpMM. In total, 1409 sparse matrices and 4 Graph-structured datasets are used in our evaluation. We compare CoTC-SpMM against the following state-of-the-art GPU SpMM implementations: ① *cuSparse*[16], NVIDIA's optimized sparse linear algebra library; ② *RoDe*[18] and *Sputnik*[8], CUDA-Core-based SpMM optimizations; ③ *DTC-SpMM*[7] and *FlashSparse*[19], TensorCore-based SpMM designs. ④ *HC-SpMM*[15], hybrid-core-based SpMM design.

4.2 Performance Evaluation

4.2.1 Workload I. We evaluate all methods in FP16 precision on both A100 and H800 GPUs. Each experiment is repeated 10 times,

and the average throughput is reported. Fig. 9 summarizes the performance on the SuiteSparse matrices. CoTC-SpMM, equipped with highly-optimized cooperative Tensor cores kernels, generally outperforms FlashSparse, achieving average speedups of 1.21 $\times$ (A100) and 1.10 $\times$ (H800), with maximum improvements of 1.93 $\times$ and 2.11 $\times$, respectively. These gains primarily result from our dual-buffer asynchronous prefetching pipeline, which effectively overlaps global-to-shared memory transfers with MMA fragment loading and computation, thereby reducing per-iteration latency and improving TC utilization. On a small subset of matrices, CoTC-SpMM is slightly slower than FlashSparse. This is because the use of **cp.async** introduces additional register pressure, causing a subset of warps within the thread block to be dedicated to data staging rather than computation, which in turn reduces available parallelism. Overall, CoTC-SpMM delivers substantial performance gains across different matrix configurations. As shown in the right-side table of Fig. 9, CoTC-SpMM consistently outperforms all baseline methods under various BN settings, demonstrating strong robustness to changes in the dense dimension. In particular, when $BN = 256$, CoTC-SpMM achieves average speedups of $3.07 \times / 2.29 \times$ over *cuSparse*, $2.76 \times / 2.10 \times$ over *Sputnik*, $2.62 \times / 2.20 \times$ over *DTC-SpMM*, and $1.78 \times / 1.25 \times$ over *RoDe* on NVIDIA A100/H800 GPU, respectively. These results confirm that CoTC-SpMM effectively scales with the dense dimension and maintains a clear advantage over state-of-the-art SpMM implementations.

4.2.2 Workload II. Artificial intelligence workloads typically rely heavily on Tensor cores for performance acceleration [22, 28]. Therefore, this section focuses on analyzing the performance of TC-based SpMM under varying sparsity levels. We evaluate 1130 sparse matrices extracted from ResNet50 using diverse pruning strategies, with matrix dimensions ranging from 64 to 2048. These matrices are generally non-square, and thus cannot be executed

²chrsmrrs.github.io/datasets/docs/datasets

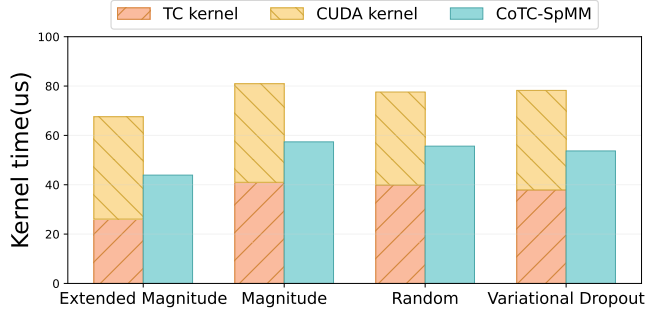


Figure 11: Runtime comparison of separate and cooperative execution strategies.

Table 2: Performance comparison of hybrid SpMM kernel libraries on graph-structured datasets.

Datasets	Edge	dim	HC-SpMM	Ours	Speedup
DD	1,686,092	89	0.40ms	0.28ms	1.42×
Yeast	3,636,546	74	1.46ms	0.63ms	2.32×
OVCAR	3,946,402	66	1.58ms	0.71ms	2.21×
YeastH	6,487,230	75	3.21ms	1.28ms	2.51×

using FlashSparse and HC-SpMM; they are therefore excluded from comparison. Fig. 10 shows the average performance across different sparsity ranges, with cuSparse used as the baseline. As a result, DTC-SpMM and TC-GNN are primarily optimized for large matrices and therefore exhibit noticeable degradation on smaller ones, often falling behind cuSparse. In contrast, CoTC-SpMM exploits cooperative parallelism between TCs and CUDA cores, enhancing the utilization of SMs. Compared with cuSparse, the average speedups of CoTC-SpMM among four distinct pruning methods can achieve 1.42× and 1.40× on A100 and H800, respectively. Moreover, the speedup generally increases with sparsity across different pruning strategies, with only minor variation due to specific sparsity patterns.

4.2.3 Workload III. Table. 2 summarizes the edge counts and node feature dimensions of the graph-structured datasets, along with the runtime obtained using different hybrid SpMM kernels. The experimental results show that CoTC-SpMM achieves average speedups of 2.11× over HC-SpMM. The performance gains are primarily attributable to two design choices. First, CoTC-SpMM utilizes the low-level mma PTX instruction, which enables finer-grained manipulation of sparse data layouts and compiles into more efficient SASS instructions. Second, CoTC-SpMM adopts a fine-grained window partitioning strategy and allocates workloads according to the performance characteristics of individual hardware cores, thereby approaching the theoretical peak performance of the SpMM kernel. These design choices not only eliminate unnecessary abstraction overhead but also adapt to hardware heterogeneity, making CoTC-SpMM a highly efficient and practical solution for SpMM acceleration.

4.2.4 Separative VS Cooperative. In order to quantify the performance gains enabled by TC-CUDA cooperation, Fig. 11 compares

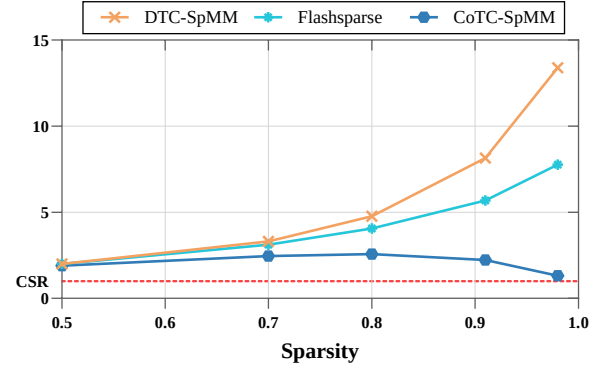


Figure 12: Storage cost comparison of sparse formats for TC-based SpMM frameworks.

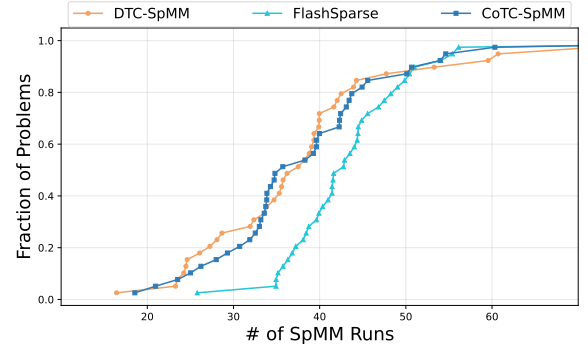


Figure 13: Preprocessing cost comparison of sparse formats for TC-based SpMM frameworks. For each point (x,y) in this plot, the cost of preprocessing is amortized after x SpMM iterations for y fraction of input problems.

the execution time of separate (serial) kernel execution with our cooperative parallel strategy. When GPU resources are sufficient and no inter-kernel dependencies exist, TC kernels and CUDA kernels can be scheduled concurrently, enabling true kernel-level parallelism. CoTC-SpMM leverages this capability by decoupling the computation mapped to TCs and CUDA cores, allowing each to process complementary portions of the workload. As a result, cooperative execution significantly reduces overall latency and delivers an average speedup of 1.46× over serial execution.

4.3 Overhead Analysis of CoTC-SpMM

4.3.1 Storage Overhead Analysis. Fig. 12 compares the storage overhead of converting unstructured sparse matrices to dense TC blocks under the TC-based SpMM frameworks, with the CSR format as the baseline. As shown in the figure, the average filling zero element within TC blocks in both DTC-SpMM and FlashSparse gradually rises as the matrix sparsity continues to increase, leading to a sharp increase in the required storage space. Moreover, the excessive involvement of such filling-zero element in MMA execution further amplifies computational redundancy. In contrast, the partitioning mechanism of the HTC sparse format ensures that CoTC-SpMM

consistently maintains a stable memory overhead. When matrices are extremely sparse, most non-zero elements are processed by CUDA cores, making the storage overhead comparable to that of the CSR format. Compared with the CSR format, CoTC-SpMM increases the average storage overhead by 1.09 $\times$, lower than DTC-SpMM 6.32 $\times$ and FlashSparse 4.52 $\times$.

4.3.2 Preprocessing Overhead of Format Conversion. Fig. 13 presents a comparison of preprocessing overhead of format conversion among various SpMM frameworks based on tensor core. Note that, the preprocessing stage of all the above frameworks is accelerated by GPU. As shown in the figure, the preprocessing overhead of CoTC-SpMM is lower than FlashSparse and roughly the same as DTC-SpMM. For example, after 40 iterations, the proportion of dataset that can evenly distribute preprocessing costs is only 33.33% for FlashSparse, while DTC-SpMM and CoTC-SpMM are 71.79% and 64.10%, respectively. The primary preprocessing overhead in TC-based SpMM frameworks comes from assembling TC blocks. Specifically, the local index of non-zero element must be determined within the sequential layout of the TC block. CoTC-SpMM reduces TC blocks assembly overhead through dense column partitioning, while directly storing sparse non-zero elements in CSR format. This makes CoTC-SpMM as a viable trade-off solution suitable for various scenarios.

5 Related Work

SpMM on CUDA core. Two aspects are considered in previous studies to optimize SpMM performance on CUDA cores: Fine-grained load balancing and pipelining technologies. Sputnik [8] executes SpMM kernel using a 1-D tiling strategy and employing ROMA to implicitly pad the rows of sparse matrices, thereby ensuring aligned data addresses for each thread block. RoDe [18] decomposes the sparse matrix into block and residual components, processes each with a separate pipeline kernel, and further proposes a block split method for load balancing.

SpMM on Tensor core. Existing optimization efforts for SpMM on tensor cores mainly focus on data format transformation and the utilization of MMA instructions for sparse data computation. Magicube [14] introduces the SR-BCRS sparse format to facilitate low-precision MMA operations on tensor cores, while SpMM kernels are executed using a 1-D block strategy. TC-GNN [26] proposes Sparse Graph Translation (SGT), which condenses the sparse matrix by assembling column fragments that contain non-zero elements to form TC blocks exhibiting dense properties. FS-AMM [24] employs a perceptron to differentiate between dense and sparse tiles within a sparse matrix, and applies the MMA.SP instruction to process 2:4 sparsity sparse tiles. MP-SpMM [6] transforms the original unstructured sparse matrix into a 2:4 structured pattern using matching and padding strategies, thereby enabling efficient computation via the MMA.SP instruction. DTC-SpMM [7] primarily focuses on optimizing runtime SpMM kernel performance through techniques such as double buffering, load balancing, register remapping, and pipeline strategies. ACC-SpMM [29] enhances the non-zero density of TC blocks by reordering the sparse matrix to concentrate non-zero elements near the diagonal. FlashSparse [19] employs a novel swap-and-transpose matrix multiplication strategy to minimize the

sparse granularity of SpMM on the TCU and reduce the number of MMA instruction invocations.

SpMM on hybrid computing core. To fully leverage the computational advantages of both CUDA cores and Tensor cores on GPUs, existing work redivides sparse matrices into dense and sparse parts and reasonably distributes the computational tasks across the different core types. BRP-SpMM [5] introduces an innovative Block-Row Partition (BRP) approach that reorganizes the sparse matrix into two distinct parts, which are assigned to two highly optimized runtime kernels for sequential execution. HC-SpMM [15] adopts the row window as the minimal unit and uses a logistic regression model to determine the appropriate GPU cores for matrix multiplication of a row window. HR-SpMM [25] partitions sparse rows into short and long rows according to the non-zero counts. The long rows are assigned to TCs, while the short rows are processed by CUDA cores. In contrast, CoTC-SpMM achieves the finest-grained partitioning by using column vectors to separate sparse and dense regions, enabling distinct kernels to execute in parallel via a collaborative scheme.

6 Conclusion

In this paper, we present CoTC-SpMM, a cooperative Tensor–CUDA core execution scheme to accelerate SpMM on GPUs. By introducing the HTC format, CoTC-SpMM reorganizes sparse matrices into Tensor Core-aligned dense blocks and CUDA-efficient residual segments, enabling optimized memory access patterns and balanced workload distribution. To accommodate diverse sparsity distributions, the framework incorporates a pair of highly optimized runtime kernels managed by a lightweight selector, further maximizing hardware parallelism. Extensive evaluations demonstrate that CoTC-SpMM consistently outperforms state-of-the-art implementations, delivering substantial speedups across both scientific computing and AI workloads. Beyond acceleration, CoTC-SpMM provides a superior trade-off by minimizing memory footprint without compromising format conversion efficiency.

Acknowledgments

We deeply appreciate the anonymous reviewers and the ICPP'2026 program committee for their invaluable comments, which have greatly contributed to improving the quality of this work. This work was supported in part by the National Key R&D Program of China (Grant No. 2023YFB3002702), in part by the Programs of the National Natural Science Foundation of China (Grant Nos. 62321003, 62502155, and 62572491), in part by the Programs of the Hunan Province (Grant Nos. 2024JJ2026, 2025RC3005, and 2026JK2074), and in part by the Program of State Key Laboratory of Parallel and Distributed Processing of China (Grant No. 2024-KJWPD-10).

References

- [1] Sergi Abadal, Akshay Jain, Robert Guirado, Jorge López-Alonso, and Eduard Alarcón. 2021. Computing graph neural networks: A survey from algorithms to accelerators. *ACM Computing Surveys (CSUR)* 54, 9 (2021), 1–38.
- [2] Hartwig Anzt, Stanimire Tomov, and Jack J Dongarra. 2015. Accelerating the LOBPCG method on GPUs using a blocked sparse matrix vector product.. In *SpringSim (HPS)*. 75–82.
- [3] Zhaodong Chen, Zheng Qu, Liu Liu, Yufei Ding, and Yuan Xie. 2021. Efficient tensor core-based gpu kernels for structured sparsity under reduced precision. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14.

- [4] Timothy A Davis and Yifan Hu. 2011. The University of Florida sparse matrix collection. *ACM Transactions on Mathematical Software (TOMS)* 38, 1 (2011), 1–25.
- [5] Yukang Dong, Wenbin Jiang, Xinhai Shen, Haihong Guo, Zhiyuan Shao, and Hai Jin. 2025. BRP-SpMM: Block-Row Partition Based Sparse Matrix Multiplication with Tensor and CUDA Cores. In *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 901–912.
- [6] Yukang Dong, Ziyuan Shen, Wenbin Jiang, Zhenghang Liu, Ye Xu, Bingyi He, Ran Zheng, and Hai Jin. 2025. Bridging the Gap between Unstructured SpMM and Structured Sparse Tensor Cores. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 645–660.
- [7] Ruibo Fan, Wei Wang, and Xiaowen Chu. 2024. Dtc-spm: Bridging the gap in accelerating general sparse matrix multiplication with tensor cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 253–267.
- [8] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse gpu kernels for deep learning. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–14.
- [9] Akram Hajiveisheh, Seyed Amjad Seyed, and Fardin Akhlaghian Tab. 2024. Deep asymmetric nonnegative matrix factorization for graph clustering. *Pattern Recognition* 148 (2024), 110179.
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [11] Changwan Hong, Aravind Sukumaran-Rajam, Bortik Bandyopadhyay, Jinsung Kim, Süreyya Emre Kurt, Israt Nisa, Shivani Sabhlok, Ümit V Çatalyürek, Srinivasan Parthasarathy, and P Sadayappan. 2018. Efficient sparse-matrix multi-vector product on gpus. In *Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing*. 66–79.
- [12] Changwan Hong, Aravind Sukumaran-Rajam, Israt Nisa, Kunal Singh, and Ponnuwamy Sadayappan. 2019. Adaptive sparse tiling for sparse matrix multiplication. In *Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming*. 300–314.
- [13] Guyue Huang, Guohao Dai, Yu Wang, and Huazhong Yang. 2020. GE-SpMM: General-Purpose Sparse Matrix-Matrix Multiplication on GPUs for Graph Neural Networks. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–12.
- [14] Shigang Li, Kazuki Osawa, and Torsten Hoeftler. 2022. Efficient quantized sparse matrix operations on tensor cores. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [15] Zhonggen Li, Xiangyu Ke, Yifan Zhu, Yunjun Gao, and Yaofeng Tu. 2025. HC-SpMM: Accelerating Sparse Matrix-Matrix Multiplication for Graphs with Hybrid GPU Cores. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 501–514.
- [16] Maxim Naumov, L Chien, Philippe Vandermersch, and Ujval Kapasi. 2010. Cusparse library. In *GPU Technology Conference*, Vol. 12.
- [17] Patrik Okanovic, Grzegorz Kwasniewski, Paolo Sylos Labini, Maciej Besta, Flavio Vella, and Torsten Hoeftler. 2024. High performance unstructured spmm computation using tensor cores. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–14.
- [18] Meng Pang, Xiang Fei, Peng Qu, Youhui Zhang, and Zhaolin Li. 2024. A row decomposition-based approach for sparse matrix multiplication on GPUs. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 377–389.
- [19] Jinliang Shi, Shigang Li, Youxuan Xu, Rongtian Fu, Xueying Wang, and Tong Wu. 2025. Flashsparse: Minimizing computation redundancy for fast sparse matrix multiplications on tensor cores. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 312–325.
- [20] Yingchen Song, Yaobin Wang, Chaoyu Xiong, Tianhai Wang, and Pingping Tang. 2024. An efficient sampling-based SpMM kernel for balancing accuracy and speed in GNN inference. In *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*. IEEE, 468–475.
- [21] Markus Steinberger, Rhaleb Zayer, and Hans-Peter Seidel. 2017. Globally homogeneous, locally adaptive sparse matrix-vector multiplication on the GPU. In *Proceedings of the International Conference on Supercomputing*. 1–11.
- [22] Yufei Sun, Long Zheng, Qinggang Wang, Xiangyu Ye, Yu Huang, Pengcheng Yao, Xiaofei Liao, and Hai Jin. 2022. Accelerating sparse deep neural network inference using gpu tensor cores. In *2022 IEEE high performance extreme computing conference (HPEC)*. IEEE, 1–7.
- [23] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. 2022. Efficient Transformers: A Survey. *ACM Comput. Surv.* 55, 6, Article 109 (2022), 28 pages.
- [24] Haotian Wang, Yan Ding, Yumeng Liu, Weichen Liu, Chubo Liu, Wangdong Yang, and Kenli Li. 2024. A context-awareness and hardware-friendly sparse matrix multiplication kernel for CNN inference acceleration. *IEEE Trans. Comput.* 74, 4 (2024), 1182–1195.
- [25] Qi Wang, Yaobin Wang, Yi Luo, Rong Luo, and Pingping Tang. 2025. Hr-spm: Adaptive row partitioning and hybrid kernel design for sparse matrix multiplication. In *Proceedings of the 39th ACM International Conference on Supercomputing*. 161–172.
- [26] Yuke Wang, Boyuan Feng, Zheng Wang, Guyue Huang, and Yufei Ding. 2023. {TC-GNN}: Bridging sparse {GNN} computation and dense tensor cores on {GPUs}. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 149–164.
- [27] Jie Xin, Xianqi Ye, Long Zheng, Qinggang Wang, Yu Huang, Pengcheng Yao, Linchen Yu, Xiaofei Liao, and Hai Jin. 2021. Fast Sparse Deep Neural Network Inference with Flexible SpMM Optimization Space Exploration. In *2021 IEEE High Performance Extreme Computing Conference (HPEC)*. 1–7.
- [28] Jianrong Yan, Wenbin Jiang, Dongao He, Suyang Wen, Yang Li, Hai Jin, and Zhiyuan Shao. 2025. RT-GNN: Accelerating sparse graph neural networks by Tensor-CUDA kernel fusion. *ACM Transactions on Architecture and Code Optimization* 22, 1 (2025), 1–27.
- [29] Haisha Zhao, San Li, Jiaheng Wang, Chunbao Zhou, Jue Wang, Zhikuan Xin, Shunde Li, Zhiqiang Liang, Zhijie Pan, Fang Liu, et al. 2025. Acc-spm: Accelerating general-purpose sparse matrix-matrix multiplication with gpu tensor cores. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 326–338.
- [30] Maohua Zhu, Tao Zhang, Zhenyu Gu, and Yuan Xie. 2019. Sparse tensor core: Algorithm and hardware co-design for vector-wise sparse neural networks on modern gpus. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 359–371.